Chapter 4 : Quantum Information Science

Gyuyoung Hwang

April 8, 2022

Seoul National University
hgy0407@snu.ac.kr

Table of Contents

Table of Contents

In this chapter, we study basic concepts to prepare our journey to the quantum machine learning.

Topics include basic algorithms, quantum parallelism, teleportation, and associated programming in Qiskit and Cirq.

Part 1 : Qiskit and Cirq to construct quantum circuits and states, Von Neumann Entropy

Part 2 : Quantum Teleportation, Gate Scheduling, Quantum Parallelism and Function Evaluation, Deutsch's Algorithm

Part 1 : Quantum Circuits with Qiskit and Cirq , and Von Neumann Entropy

Table of Contents

Quantum states and Observables: A review

A pure state is a vector $|\psi\rangle$ in a Hilbert space $\mathbb{H}$.

In our case, $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$.

An observable A is a self-adjoint operator on $\mathbb{H}$.

We compute the mean value of A by $\langle A\rangle_\psi := \langle\psi|A\psi\rangle$.

The probability $P_\psi(\lambda)$ that for a quantum system in the pure state $|\psi\rangle \in \mathbb{H}$, a measurement of the observable yields the eigenvalue λ of A is given by

$$P_\psi(\lambda) = \|P_\lambda|\psi\rangle\|^2,$$

where P_λ is the projection onto the eigenspace $Eig(A, \lambda)$.

Quantum states and Observables: A review

Projection postulate : If a measurement of the observable A on a quantum mechanical system in the pure state $|\psi\rangle \in \mathbb{H}$ yields the eigenvalue λ, then

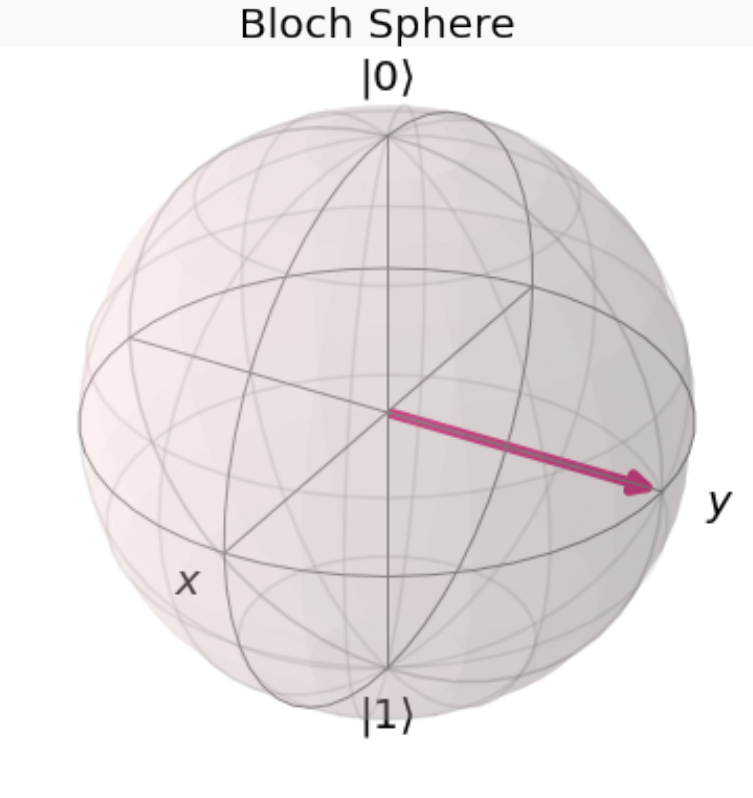
$$|\psi\rangle = \text{state before measurement} \rightarrow \frac{P_\lambda|\psi\rangle}{\|P_\lambda|\psi\rangle\|} = \text{state after measurement.}$$

A density operator is an operator ρ acting on $\mathbb{H}$ which is self-adjoint, postive($\rho \geq 0$), and has trace 1. Then, the quantum states described by density operators are called mixed states.
Indeed, we can write

$$\rho = \sum_j p_j |\psi_j\rangle\langle\psi_j|$$

Bloch representation of pure and mixed states

Since a qubit $|\psi\rangle \in \mathbb{HS}^1$, we can represent it on a sphere.



Bloch representation of pure and mixed states

To be more specific, $|\psi\rangle = a|0\rangle + b|1\rangle$ with $|a|^2 + |b|^2 = 1$ allows a parametrization

$$|\psi\rangle = e^{-\mathrm{i}\frac{\phi}{2}} \cos\frac{\theta}{2}|0\rangle + e^{\mathrm{i}\frac{\phi}{2}} \sin\frac{\theta}{2}|1\rangle.$$

For a density operator

$$\rho = \begin{pmatrix} a & b \\ c & d, \end{pmatrix}$$

using $\rho^* = \rho$ and $tr(\rho) = 1$, we get

$$\rho = \frac{1}{2}(1 + x \cdot \sigma) = \frac{1}{2}\begin{pmatrix} 1 + x_3 & x_1 - \mathrm{i}x_2 \\ x_1 + \mathrm{i}x_2 & 1 - x_3 \end{pmatrix},$$

where $\sigma = (\sigma_x, \sigma_y, \sigma_z)$.

Table of Contents

Getting started with python libraries

Qiskit : Library from IBM

Cirq : Library from Google

Getting started with Qiskit

Good introduction and guide for using the Qiskit.

How to Install Qiskit – Programming on Quantum Computers Season 1 Ep 2

Getting started with Qiskit

Anaconda promt ⇒ pip install qiskit

```
(base) C:¥Users¥admin>pip install qiskit
Requirement already satisfied: qiskit in c:¥users¥admin¥anaconda3¥lib¥site-packages (0.35.0)
Requirement already satisfied: qiskit-ignis==0.7.0 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit) (0.7.0)
Requirement already satisfied: qiskit-aer==0.10.3 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit) (0.10.3)
Requirement already satisfied: qiskit-terra==0.20.0 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit) (0.20.0)

Requirement already satisfied: qiskit-ibmq-provider==0.18.3 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit)
(0.18.3)
Requirement already satisfied: numpy>=1.16.3 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-aer==0.10.3->qis
kit) (1.20.3)
Requirement already satisfied: scipy>=1.0 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-aer==0.10.3->qiskit
) (1.7.1)
Requirement already satisfied: requests-ntlm>=1.1.0 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-ibmq-prov
ider==0.18.3->qiskit) (1.1.0)
Requirement already satisfied: urllib3>=1.21.1 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-ibmq-provider=
=0.18.3->qiskit) (1.26.7)
Requirement already satisfied: requests>=2.19 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-ibmq-provider==
0.18.3->qiskit) (2.26.0)
Requirement already satisfied: python-dateutil>=2.8.0 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-ibmq-pr
ovider==0.18.3->qiskit) (2.8.2)
Requirement already satisfied: websocket-client>=1.0.1 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-ibmq-p
rovider==0.18.3->qiskit) (1.3.2)
Requirement already satisfied: retworkx>=0.8.0 in c:¥users¥admin¥anaconda3¥lib¥site-packages (from qiskit-ignis==0.7.0->
```

Getting started with Qiskit

In [31]:
```
import qiskit
import matplotlib.pyplot
from math import pi
from qiskit.visualization import plot_bloch_vector, plot_bloch_multivector
```

In [2]:
```
qiskit.__qiskit_version__
```

```
{'qiskit-terra': '0.20.0', 'qiskit-aer': '0.10.3', 'qiskit-ignis': '0.7.0', 'qiskit-ibmq-provider': '0.18.3', 'qiskit-aqua': None,
'qiskit': '0.35.0', 'qiskit-nature': None, 'qiskit-finance': None, 'qiskit-optimization': None, 'qiskit-machine-learning': None}
```

In [3]:
```
from qiskit import IBMQ
```

Getting started with Qiskit

```
from qiskit import IBMQ

IBMQ.save_account(
```


```

IBMQ.load_account()
```

Drawing Bloch sphere using Qiskit

```
plot_bloch_vector([0,1,0], title = "Bloch Sphere")
```

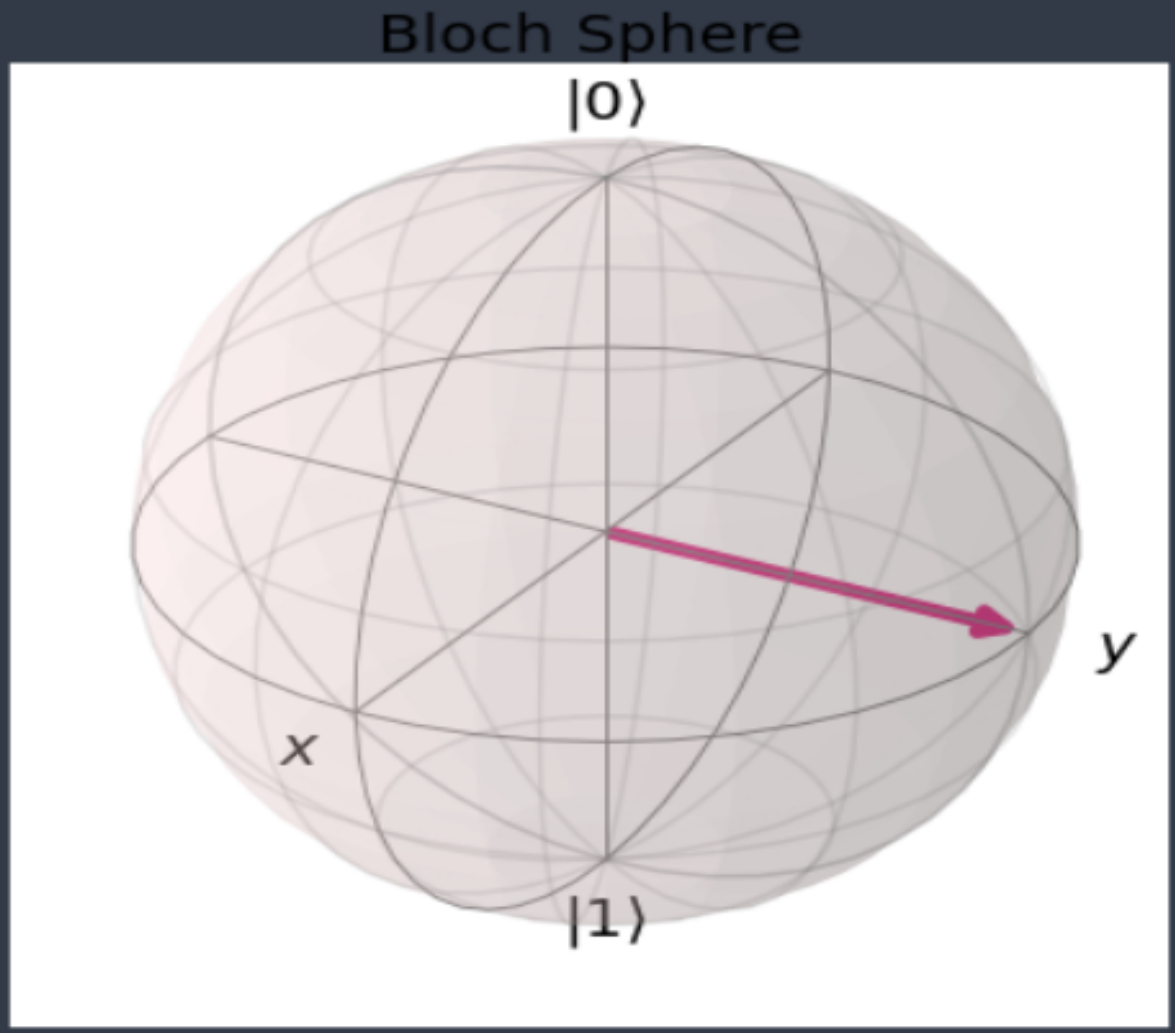


Table of Contents

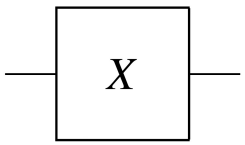
Quantum Circuits : A review

Definition

A quantum gate is a unitary transformation $\mathcal{U} : \mathbb{H}^{\otimes n} \to \mathbb{H}^{\otimes n}$.
If $n = 1$, we call it unary, and if $n = 2$, we call it binary.

Three representations of a quantum gate: Operator, matrix, and symbol.

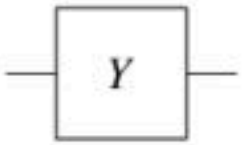
Examples of unary transformation : $V : \mathbb{H} \to \mathbb{H}$, unitary 2×2 matrix.
1. Pauli-X gate (Or Q-Not gate)



$$X := \sigma_x \qquad \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

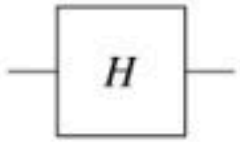
Quantum Circuits : A review

2. Pauli-Y gate



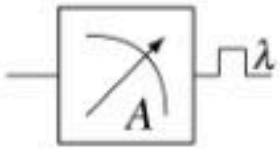
$$Y := \sigma_y \qquad \begin{pmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{pmatrix}$$

3. Hadamard gate



$$H := \frac{\sigma_x + \sigma_z}{\sqrt{2}} \qquad \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

4, Measurement of observable A (this is not a gate).



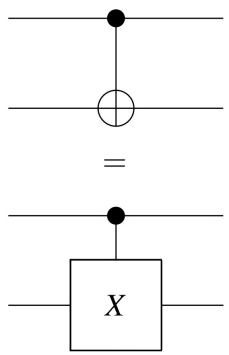
[Not a gate, but a non-unitary transformation of the input-state to an eigenstate of A and delivery of measured value λ]

Quantum Circuits : A review

Examples of binary quantum gate : $\mathcal{U} : \mathbb{H}^{\otimes 2} \to \mathbb{H}^{\otimes 2}$ with basis $\{|0\rangle^2, |1\rangle^2, |2\rangle^2, |3\rangle^2\}$.

1. $\Lambda^1(X)$

Controlled NOT (C-NOT)

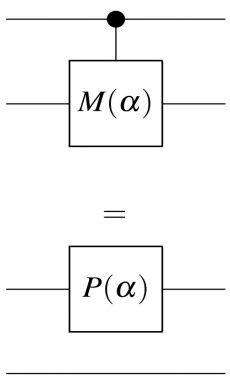


$$\Lambda^1(X) = |0\rangle\langle 0| \otimes \mathbf{1} + |1\rangle\langle 1| \otimes X$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

2.

Controlled phase-multiplication



$$\Lambda^1(M(\alpha)) = |0\rangle\langle 0| \otimes \mathbf{1} + |1\rangle\langle 1| \otimes e^{i\alpha}\mathbf{1} = P(\alpha) \otimes \mathbf{1}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & e^{i\alpha} & 0 \\ 0 & 0 & 0 & e^{i\alpha} \end{pmatrix}$$

Quantum Circuits : A review

Definition

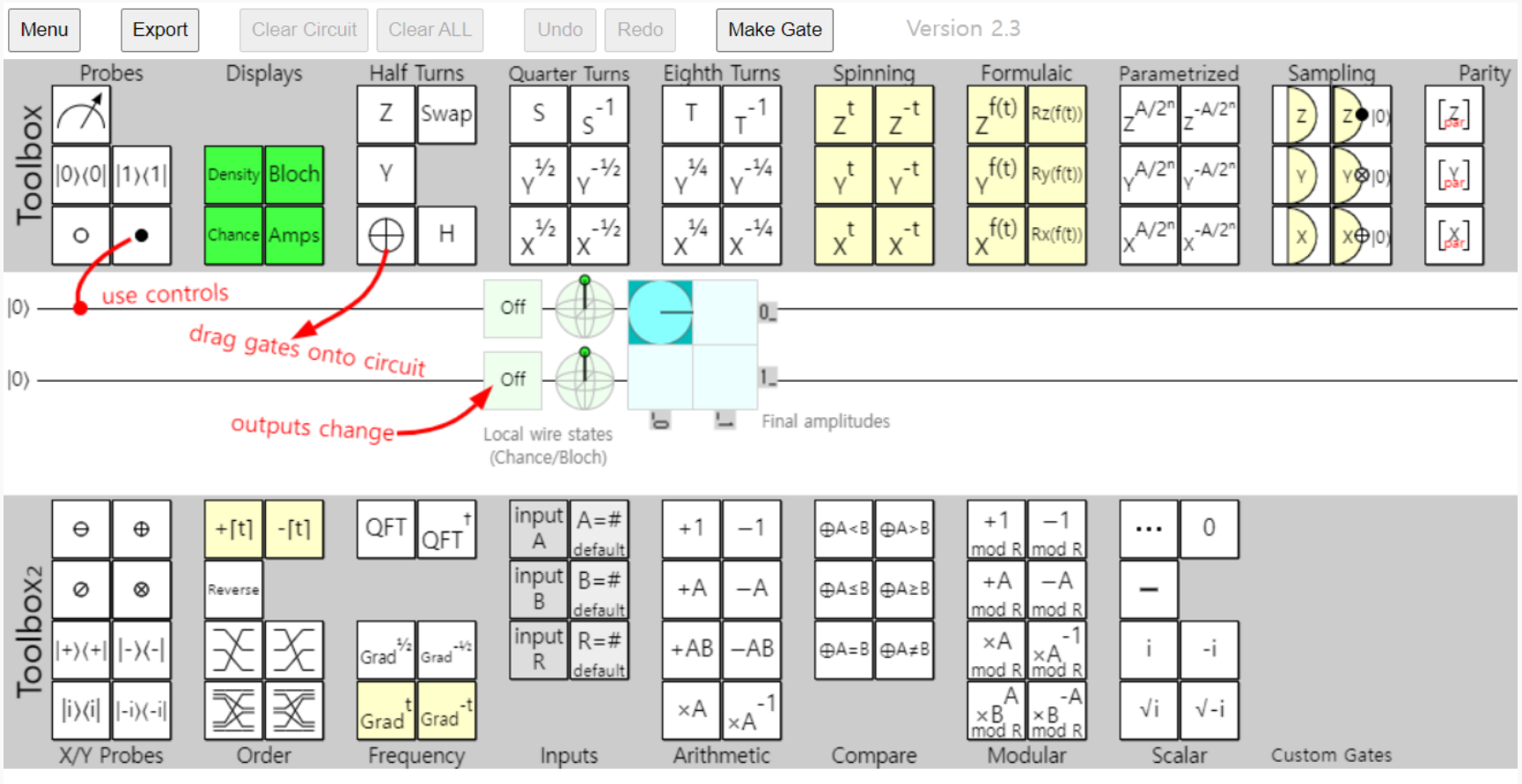
Let $n, L \in \mathbb{N}$ and let $\mathcal{U}_1, \cdots, \mathcal{U}_L \in \mathcal{U}(\mathbb{H}^{\otimes n})$ be quantum n-gates. Then the composition operator $U = \mathcal{U}_L \cdots \mathcal{U}_1$ is called the quantum circuit of depth L.

Remark : There are two other definitions of the quantum circuit. See our previous seminar book on the quantum computation.

Table of Contents

However, before those... let's play with

https://algassert.com/quirk



Getting started with Cirq

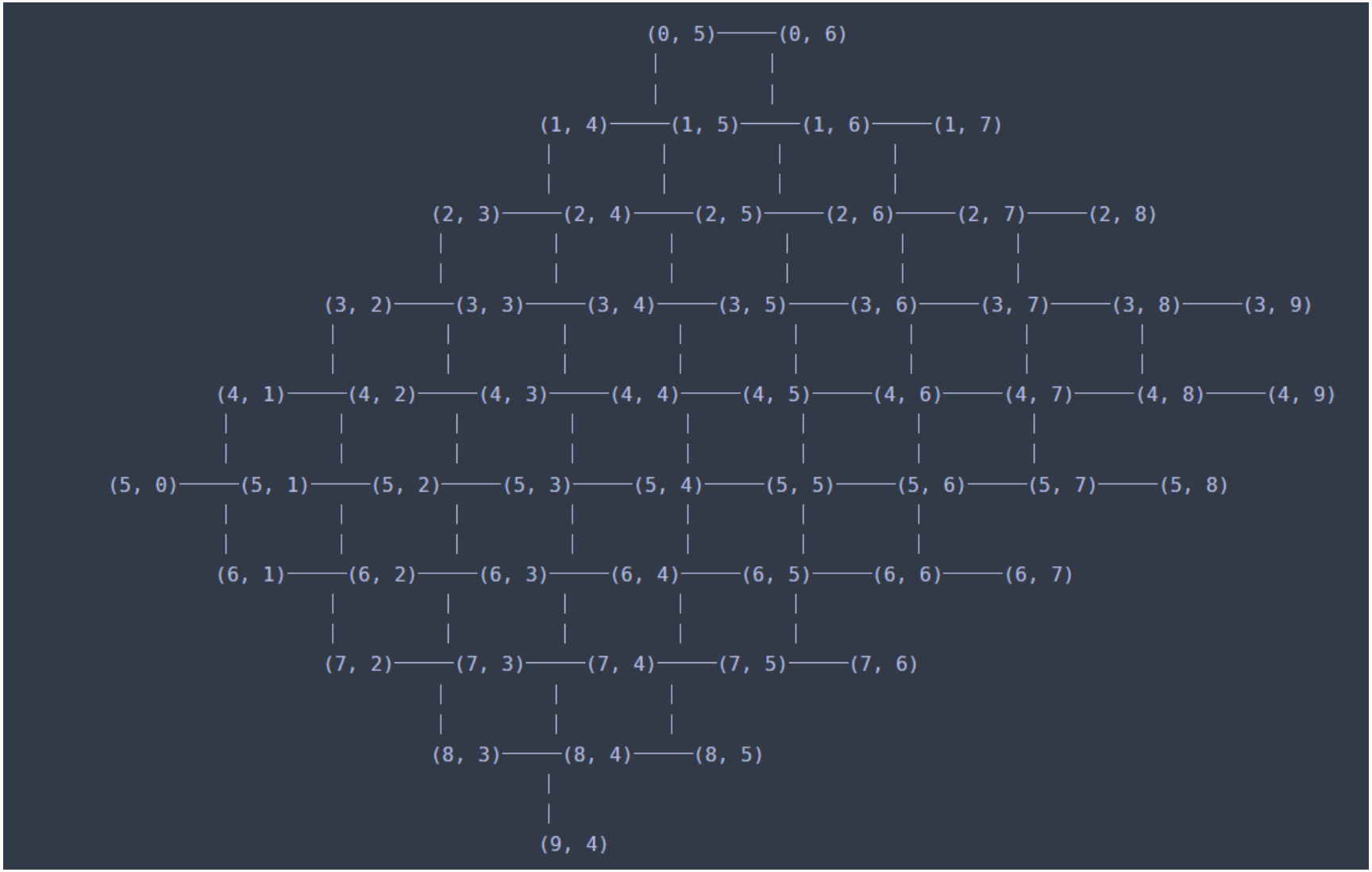
Please see : https://www.qmunity.tech/tutorials/introduction-to-cirq

```
In [ ]: pip install cirq
```

```
In [1]: import cirq
```

```
In [2]: import cirq
        import cirq_google
```

```
In [3]: print(cirq_google.Sycamore)   #Not Bristlecone it is deprecated
```


(0, 5)——(0, 6)
(1, 4)——(1, 5)——(1, 6)——(1, 7)
(2, 3)——(2, 4)——(2, 5)——(2, 6)——(2, 7)——(2, 8)
(3, 2)——(3, 3)——(3, 4)——(3, 5)——(3, 6)——(3, 7)——(3, 8)——(3, 9)
(4, 1)——(4, 2)——(4, 3)——(4, 4)——(4, 5)——(4, 6)——(4, 7)——(4, 8)——(4, 9)
(5, 0)——(5, 1)——(5, 2)——(5, 3)——(5, 4)——(5, 5)——(5, 6)——(5, 7)——(5, 8)
(6, 1)——(6, 2)——(6, 3)——(6, 4)——(6, 5)——(6, 6)——(6, 7)
(7, 2)——(7, 3)——(7, 4)——(7, 5)——(7, 6)
(8, 3)——(8, 4)——(8, 5)
(9, 4)

Creating qubits

In [12]:

```
q0=cirq.NamedQubit("q0")
q1=cirq.NamedQubit("q1")
print(q0, q1)
# Line qubits
q3 = cirq.LineQubit(4)
print(q3)
# Creates LineQubit(0), LineQubit(1), LineQubit(2)
q0, q1, q2 = cirq.LineQubit.range(3)
print(q0,q1,q2)
# Grid Qubits can also be referenced individually
q0_1= cirq.GridQubit(0,1)
print(q0_1)
# Create 16 qubits from (0,0) to (7,7)
qubits = cirq.GridQubit.square(8)
print(qubits)
```

Creating qubits : Result

```
q0 q1
4
0 1 2
(0, 1)
[cirq.GridQubit(0, 0), cirq.GridQubit(0, 1), cirq.GridQubit(0, 2), cirq.GridQubit(0, 3), cirq.GridQubit(0, 4), cirq.GridQubit(0, 5),
cirq.GridQubit(0, 6), cirq.GridQubit(0, 7), cirq.GridQubit(1, 0), cirq.GridQubit(1, 1), cirq.GridQubit(1, 2), cirq.GridQubit(1, 3),
cirq.GridQubit(1, 4), cirq.GridQubit(1, 5), cirq.GridQubit(1, 6), cirq.GridQubit(1, 7), cirq.GridQubit(2, 0), cirq.GridQubit(2, 1),
cirq.GridQubit(2, 2), cirq.GridQubit(2, 3), cirq.GridQubit(2, 4), cirq.GridQubit(2, 5), cirq.GridQubit(2, 6), cirq.GridQubit(2, 7),
cirq.GridQubit(3, 0), cirq.GridQubit(3, 1), cirq.GridQubit(3, 2), cirq.GridQubit(3, 3), cirq.GridQubit(3, 4), cirq.GridQubit(3, 5),
cirq.GridQubit(3, 6), cirq.GridQubit(3, 7), cirq.GridQubit(4, 0), cirq.GridQubit(4, 1), cirq.GridQubit(4, 2), cirq.GridQubit(4, 3),
cirq.GridQubit(4, 4), cirq.GridQubit(4, 5), cirq.GridQubit(4, 6), cirq.GridQubit(4, 7), cirq.GridQubit(5, 0), cirq.GridQubit(5, 1),
cirq.GridQubit(5, 2), cirq.GridQubit(5, 3), cirq.GridQubit(5, 4), cirq.GridQubit(5, 5), cirq.GridQubit(5, 6), cirq.GridQubit(5, 7),
cirq.GridQubit(6, 0), cirq.GridQubit(6, 1), cirq.GridQubit(6, 2), cirq.GridQubit(6, 3), cirq.GridQubit(6, 4), cirq.GridQubit(6, 5),
cirq.GridQubit(6, 6), cirq.GridQubit(6, 7), cirq.GridQubit(7, 0), cirq.GridQubit(7, 1), cirq.GridQubit(7, 2), cirq.GridQubit(7, 3),
cirq.GridQubit(7, 4), cirq.GridQubit(7, 5), cirq.GridQubit(7, 6), cirq.GridQubit(7, 7)]
```

Creating unitary matrices

```
In [5]: print(cirq.unitary(cirq.X)) #Unitary of the X gate
```

```
[[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
```

```
In [6]: a, b = cirq.LineQubit.range(2) #Unitary of SWAP operator on two qubits
        print(cirq.unitary(cirq.SWAP(a, b)))
```

```
[[1.+0.j 0.+0.j 0.+0.j 0.+0.j]
 [0.+0.j 0.+0.j 1.+0.j 0.+0.j]
 [0.+0.j 1.+0.j 0.+0.j 0.+0.j]
 [0.+0.j 0.+0.j 0.+0.j 1.+0.j]]
```

Creating Circuits

In [49]:
```
circuit=cirq.Circuit(cirq.H(q0), cirq.CNOT(q0,q1), cirq.measure(q0,q1))

print(circuit)
```

```
0: ———H———@———M———
          |   |
1: ———————X———M———
```

In [53]:
```
simulator=cirq.Simulator()

result=simulator.run(circuit,repetitions=30)
print(result)
```

```
0,1=10010110100111111011001111011, 10010110100111111011001111011
```

Creating the Bell state

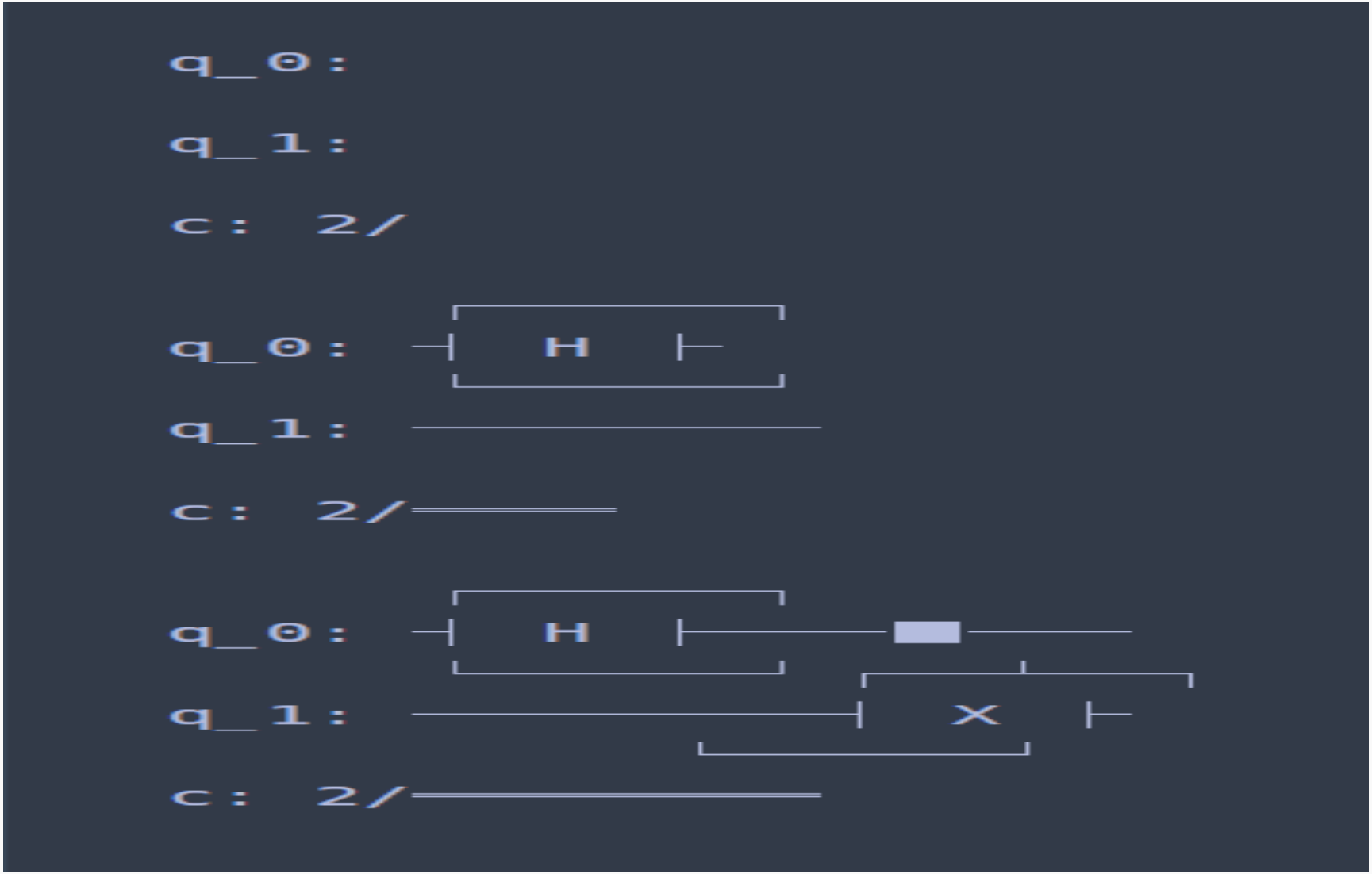
```
# We will generate the Bell State:
# sqrt(2) * ( |00> + |11> )
bell_circuit = cirq.Circuit()
a, b = cirq.LineQubit.range(2)
bell_circuit.append(cirq.H(a))
bell_circuit.append(cirq.CNOT(a,b))
print(bell_circuit)
# Initialize Simulator
s=cirq.Simulator()

print('Simulate the circuit:')
results=s.simulate(bell_circuit)
print(results)
print()

# For sampling, we need to add a measurement at the end
bell_circuit.append(cirq.measure(a, b, key='result'))

print('Sample the circuit:')
samples=s.run(bell_circuit, repetitions=1000)
```

Creating the Bell state : The result

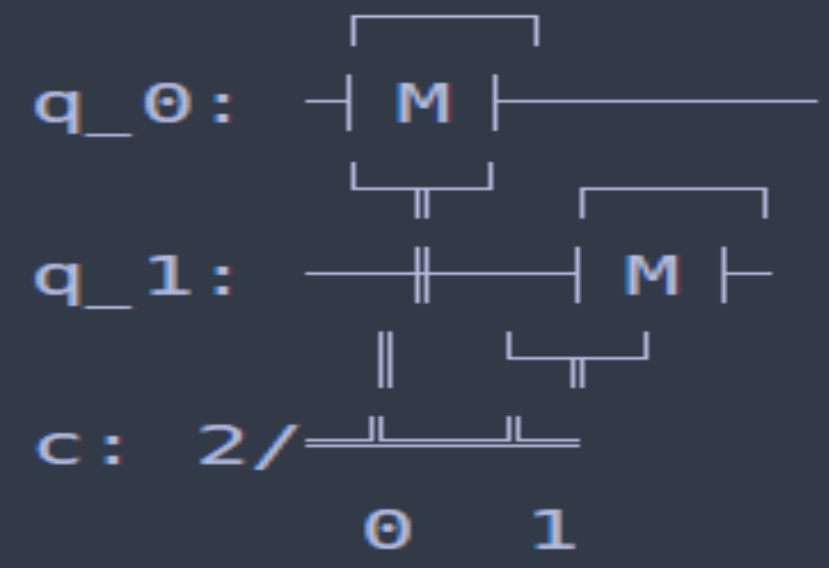
```
0: ──────H──────@──────
                │
1: ─────────────X──────
Simulate the circuit:
measurements: (no measurements)

qubits: (cirq.LineQubit(0), cirq.LineQubit(1))
output vector: 0.707|00⟩ + 0.707|11⟩

phase:
output vector: |⟩

Sample the circuit:
```

Qiskit again, Bell state again

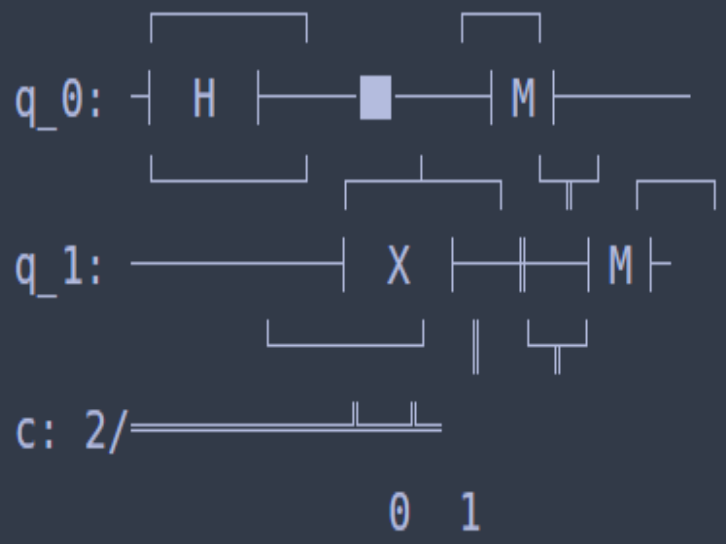
```
#quantum circuit to create a Bell state
bell = QuantumCircuit(2,2)
print(bell)
bell.h(0)
print(bell)
bell.cx(0,1)
print(bell)
```


q_0:
q_1:
c: 2/
q_0:
H
q_1:
c: 2/
q_0:
H
q_1:
X
c: 2/

```
meas = QuantumCircuit(2,2)
meas.measure([0,1],[0,1])
print(meas)
```



```
#execute the quantum circuit
backend = BasicAer.get_backend('qasm_simulator') #the device the run on
circ = bell + meas
print(circ)
```



```
result = execute(circ, backend, shots = 1000).result()
counts = result.get_counts(circ)
print(counts)
```

```
{'11': 521, '00': 479}
```

```
from qiskit.visualization import plot_histogram
plot_histogram(counts)
```

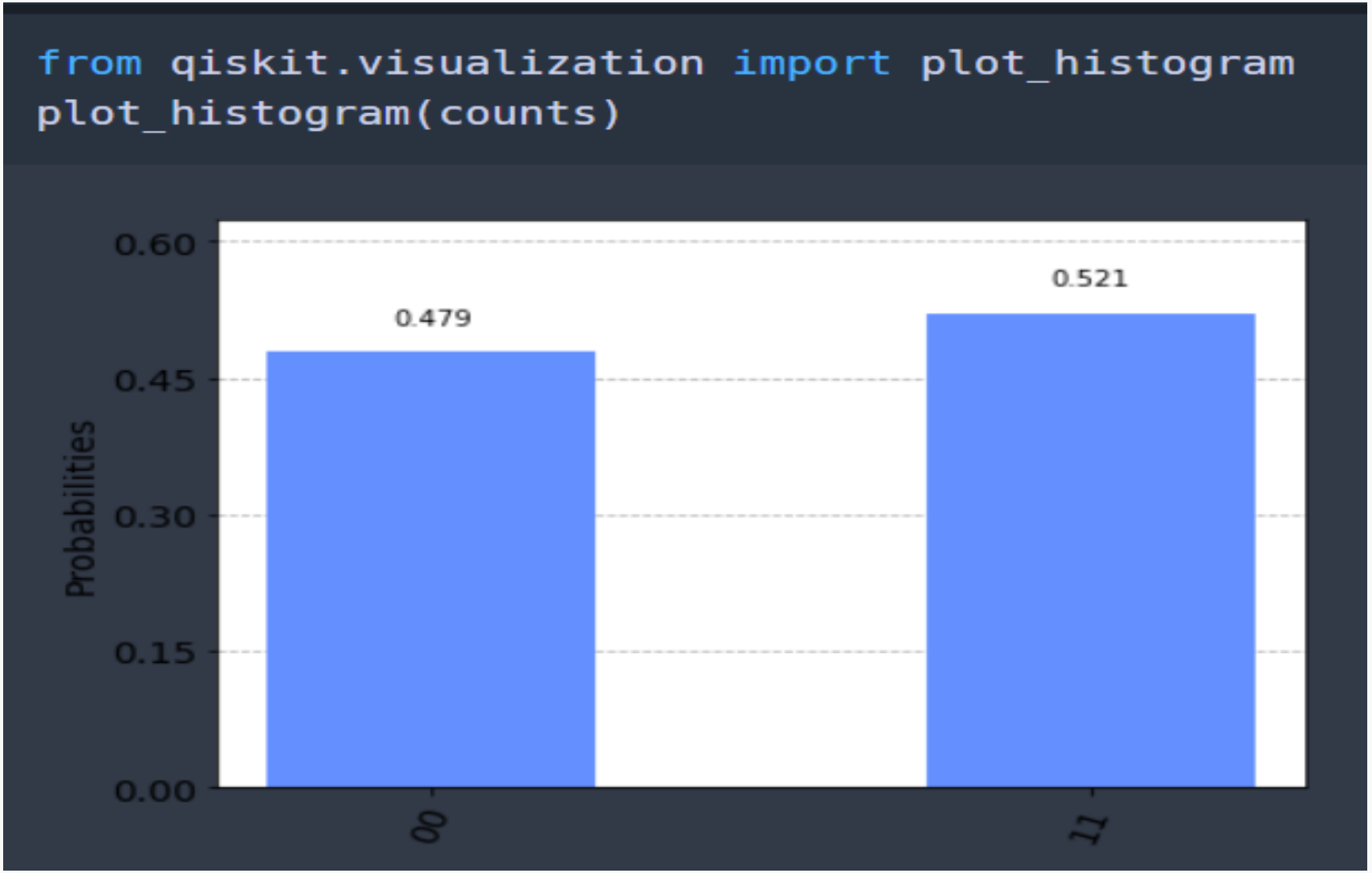


Table of Contents

Von Neumann Entropy

Quantum counterpart of the Shannon entropy

Definition

Given a mixed state $\rho = \sum_{i=1}^{n} p_i |\psi_i\rangle\langle\psi_i|$, the von Neumann entropy is given by

$$S(\rho) = -Tr(\rho \log \rho)$$

It is...

1. subadditive, $S(\rho_{12}) \leq S(\rho_1) + S(\rho_2)$.
2. $0 \leq S \leq N = dim(\mathbb{H})$.

Using Qiskit, one can compute the evolution of the von Neumann entropy of a quantum state.

see you next week :)